REQ 1
A: Quota System
Design 1: Singleton QuotaManager, there is one shared instance which is accessed via getInstance() by any class that requires the quota access.

Pros	Cons
There is no constructor pollution and satisfies KISS.	Difficult to unit test in isolation because it's a global state.
Quota logic lives in one place therefore meets SRP.	Must call resetInstance() between games.
There is low connascence of position because you do not need to know the 2nd and 3rd argument rather with the singleton you just call getInstance().	

Design 2: Pass QuotaManager as a constructor parameter, therefore every class needs quota access and will take the QuotaManager argument.

Pros	Cons
Easier to unit test with mock instances therefore better for DIP.	Clutters the constructors across many classes therefore violates KISS.
	High connascence of position because callers must know the argument order.
	The QuotaManager type spreads across many constructors therefore connascence of type

B: Quota Event Notification
Design 1: Observer pattern via the QuotaListener interface.

Pros	Cons
New reactors can be added without editing the QuotaManager therefore meets OCP.	There are more classes to understand therefore KISS.
QuotaManager will depend on the interface and not the SuperComputer therefore DIP.	
QuotaManager only manages the state and reactions belong to the listeners therefore SRP.	
Since the callers only depend on method names therefore connascence of name.	

Design 2: Direct reference to Supercomputer inside QuotaManager

Pros	Cons
Simpler and no interface needed KISS in the short term.	Every new reactor will require an editing QuotaManager therefore a OCP violation.
	QuotaManager will not know about the Supercomputer therefore SRP violation.
	QuotaManager couple to the concrete Supercomputer type therefore connascence of type.

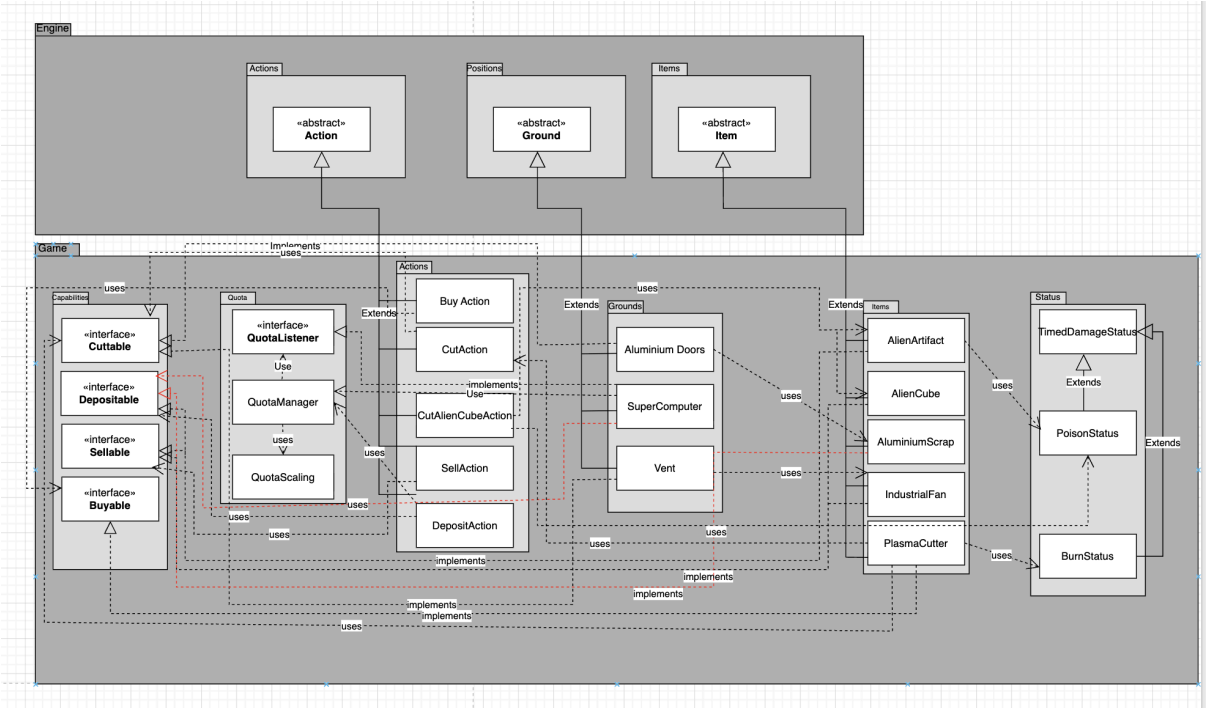
C: Deposit Capability
Design 1: Separate Depositable interface mirroring Sellable

Pros	Cons
Items will only be implemented when they actually need. Therefore meets ISP.	One more interface to maintain therefore potentially violates KISS.
DepositAction depends on the interface not on the concrete items DIP.	
Separates the worker credits from the company credits therefore meets SRP.	
Mirror the existing Sellable/SellAction pattern therefore meets DRY.	

Design 2: Add deposit method directly to Sellable

Pros	Cons
Fewer interfaces therefore meet KISS.	Forces the AluminiumScrap to implement onSold with no behaviour therefore it is a ISP violation
	Mixes worker credits with the company credits in one interface SRP violation
	Callers must know which method applies to which item therefore connascence of meaning.

UML Req 4



The UML diagram for REQ 1 introduces a Quota System, the Plasma Cutter and the resources which are required to meet the company quota. The quota system is managed by three classes including QuotaManager, QuotaScaling and QuotaListener interface. For the quota state we chose Design 1 which is the singleton QuotaManager. This is because there is one shared instance accessed via getInstance() by any class that requires the quota access. This was decided because it avoids constructor pollution within the codebase and satisfies KISS. The Quota logic in one place satisfies SRP as well, additionally there is low connascence of position since the callers do not need to know the argument order they just call getInstance(). However, this is harder to unit test in isolation. We rejected Design 2 which passes QuotaManager as a constructor parameter because it violates KISS and introduces high connascence of position as the callers must know the exact argument order.

For the quota event notification we chose Design 1 where the observer pattern is observed via the QuotaListener interface. The QuotaManager holds a list of the listeners and notified them when the quota is met or the deadline is missed. Supercomputer implements this interface shown by the realisation arrow in the diagram. This satisfies OCP since the new reactors can be added without having to edit the QuotaManager and this also satisfies DIP since the QUotaManager depends on the interface and not on the concrete Supercomputer class. SRP is met since the QuotaManager only manages state while all the reactions belong to the listeners. We rejected Design 2 with the direct reference to Supercomputer inside the QuotaManager because every new reactor would require editing of the QuotaManager which violates OCP. The QuotaManager would know about the Supercomputer therefore violating SRP and the QuotaManager would be coupled to the Supercomputer class therefore creating connascence of type.

The capabilities package contains the four interfaces Cuttable, Depositable, Sellable and Buyable. For the deposit capability we chose Design 1 which is a separate Depositable interface which mirrors Sellable and the ITem only implements what they actually need

therefore satisfying ISP. This is because some items implement sellable and depositable while others only implement depositable. The system separates the worker credits and the company credits therefore satisfies SRP. We rejected design 2 which adds deposit methods directly to Sellable because it would force AluminiumScrap to implement the onSold method with no behavior therefore violates ISP.